Solution: Triplet Sum to Zero

Problem Statement

Given an array of unsorted numbers, find all unique triplets in it that add up to zero.

Example 1:

```
Input: [-3, 0, 1, 2, -1, 1, -2]
Output: [[-3, 1, 2], [-2, 0, 2], [-2, 1, 1], [-1, 0, 1]]
Explanation: There are four unique triplets whose sum is equal to zero.
smallest sum.'
```

Example 2:

```
Input: [-5, 2, -1, -2, 3]
Output: [[-5, 2, 3], [-2, -1, 3]]
Explanation: There are two unique triplets whose sum is equal to zero.
```

Solution

This problem follows the **Two Pointers** pattern and shares similarities with Pair with Target Sum. A couple of differences are that the input array is not sorted and instead of a pair we need to find triplets with a target sum of zero.

To follow a similar approach, first, we will sort the array and then iterate through it taking one number at a time. Let's say during our iteration we are at number 'X', so we need to find 'Y' and 'Z' such that $X + Y + Z == 0$. At this stage, our problem translates into finding a pair whose sum is equal to "-X" (as from the above equation $Y + Z == -X$).

Another difference from `Pair with Target Sum` is that we need to find all the unique triplets. To handle this, we have to skip any duplicate number. Since we will be sorting the array, so all the duplicate numbers will be next to each other and are easier to skip.

Here is the detailed walk through of the algorithm:

1. The method `searchTriplets` is the main method that orchestrates the process. The algorithm starts by sorting the input array.

2. It then starts iterating over the array. For each element, it calls the method `searchPair` to find pairs in the rest of the array that add up to the negative value of the current element. The purpose of this is to find three numbers that add up to zero (i.e., the current number and two other numbers that add up to the negative of the current number).

3. The `searchPair` method uses the two-pointer technique to find pairs that add up to a given target. It starts with one pointer at the beginning of the array (or the index after the current number) and another pointer at the end of the array. It calculates the sum of the numbers at the pointers, and if the sum is equal to the target, it adds a triplet consisting of the negative target (which is the number we're currently processing in the `searchTriplets` method) and the two numbers that make up the sum to the result list.

4. If the sum is less than the target, it means we need to increase the sum, so we move the left pointer one step to the right. If the sum is greater than the target, it means we need to decrease the sum, so we move the right pointer one step to the left.

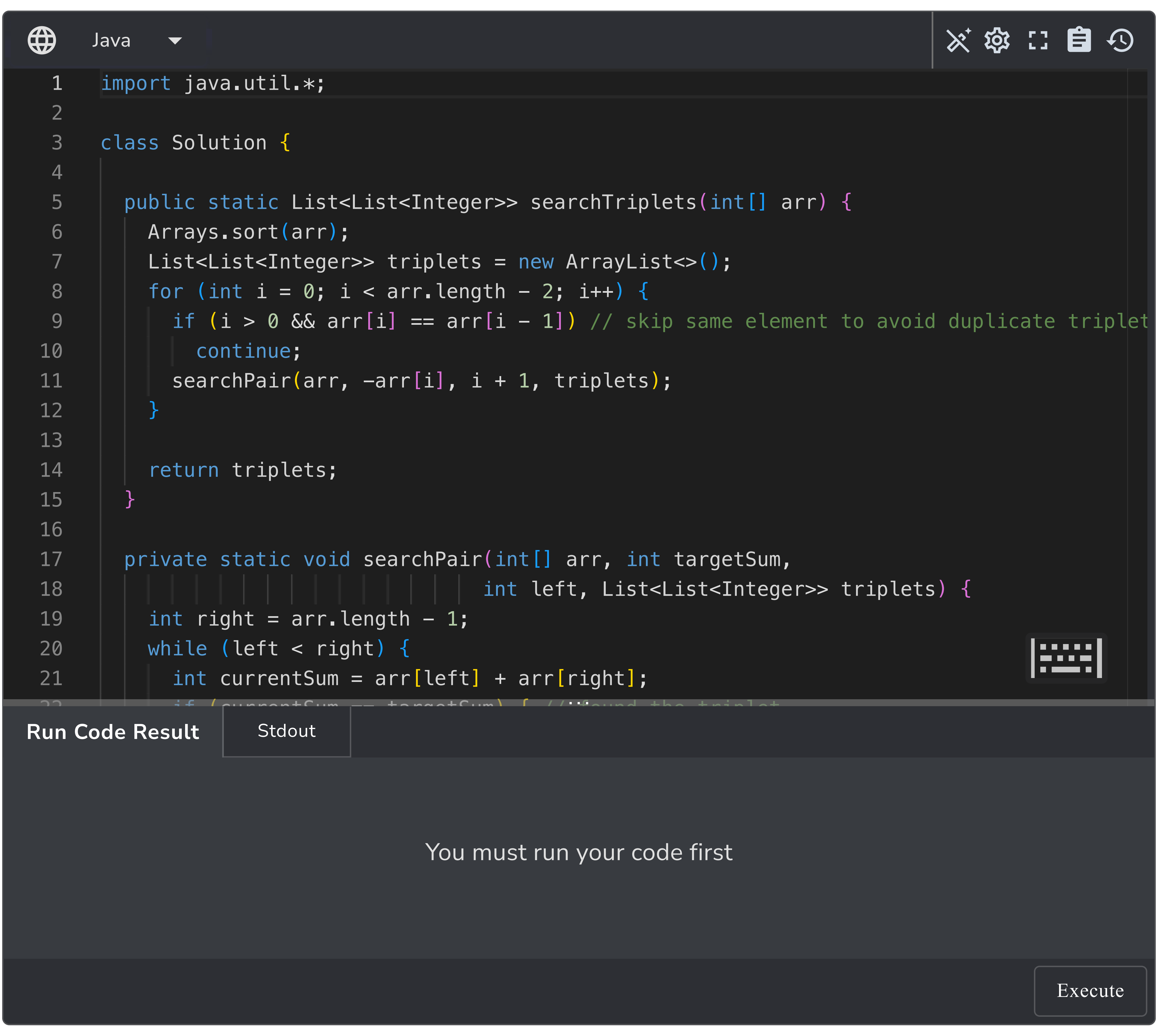
5. To avoid adding duplicate triplets to the list, the algorithm skips elements that are the same as the previous element both in the `searchTriplets` method and in the `searchPair` method.

In the end, the `searchTriplets` method returns a list of all unique triplets in the array that add up to zero.

Let's visualize the example 2 via below diagram.

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {

  public static List<List<Integer>> searchTriplets(int[] arr) {
    Arrays.sort(arr);
    List<List<Integer>> triplets = new ArrayList<>();
    for (int i = 0; i < arr.length - 2; i++) {
      if (i > 0 && arr[i] == arr[i - 1]) // skip same element to avoid duplicate triplet
        continue;
      searchPair(arr, -arr[i], i + 1, triplets);
    }

    return triplets;
  }

  private static void searchPair(int[] arr, int targetSum,
                                 int left, List<List<Integer>> triplets) {
    int right = arr.length - 1;
    while (left < right) {
      int currentSum = arr[left] + arr[right];
```

You must run your code first

Time Complexity

Sorting the array will take $O(N * logN)$. The searchPair() function will take $O(N)$. As we are calling searchPair() for every number in the input array, this means that overall searchTriplets() will take $O(N * logN + N^2)$, which is asymptotically equivalent to $O(N^2)$.

Space Complexity

Ignoring the space required for the output array, the space complexity of the above algorithm will be $O(N)$ which is required for sorting.

Please wait! Loading...

← Previous
Triplet Sum To Zero (Medium)
Next →
Triplet Sum Close To Target (Medium)
☐ Mark as Completed